

C# Assignment-05 part-01

Part 01 : Theoretical Questions

Q1 : What is an interface in C#? Why do we use interfaces instead of depending on concrete classes directly? Mention at least three benefits of using interfaces.

Answer:

Definition: Contract defining methods/properties without implementation.

Why use them: Decouple code from concrete implementations.

3 Benefits:

1. Loose coupling/testability
2. Multiple inheritance support
3. Polymorphism/flexibility

Also ability to extend for example Ipayment interface we put method pay then we extend IfawryPayment or InstaPayPayment then we override pay method and use interface to do so.

Q2 : Look at the following code and answer the questions below:

```
interface IEnglishSpeaker
{
    void Greet();
}

interface IArabicSpeaker
{
    void Greet();
}

class Translator : IEnglishSpeaker, IArabicSpeaker
{
    public void Greet()
    {
        Console.WriteLine("Hello / Ahlan");
    }
}
```

- a) What is the problem with this design? Both interfaces have a method called Greet() — how does the class handle it currently?
- b) How would you fix this so IEnglishSpeaker.Greet() says "Hello" and IArabicSpeaker.Greet() says "Ahlan"? What is this technique called?
- c) After applying your fix, can you call Greet() directly on a Translator object (e.g. translator.Greet())? Why or why not? How do you call each version?

Answers

a) Problem: Both Greet() methods map to single implementation → outputs "Hello / Ahlan" for both interfaces.

b) Fix: Use explicit interface implementation:

```
void IEnglishSpeaker.Greet() => Console.WriteLine("Hello");
```

```
void IArabicSpeaker.Greet() => Console.WriteLine("Ahlan");
```

c) Direct call? No—translator.Greet() won't compile. Must cast:

```
((IEnglishSpeaker)translator).Greet() or ((IArabicSpeaker)translator).Greet().
```

Q3 : Explain the difference between a shallow copy and a deep copy. When would you use each one? What is the risk of using a shallow copy when the object has reference-type fields?

- Shallow copy: Copies values; references point to same objects.
- Deep copy: Recursively copies all nested objects (independent clones).
- Use shallow: When object has only value types or immutable refs.
- Use deep: When you need complete independence.
- Risk: Modifying referenced objects affects both copies (shared state bug).

Q4 : Look at the following code and determine the output. Explain why.

```
class Department { public string Name; }
class Employee
{
    public string Title;
    public Department Dept;
    public Employee ShallowCopy() => (Employee)this.MemberwiseClone();
}

var e1 = new Employee { Title = "Dev", Dept = new Department { Name = "IT" } };
var e2 = e1.ShallowCopy();
e2.Title = "QA";
e2.Dept.Name = "Testing";

Console.WriteLine($"{e1.Title} - {e1.Dept.Name}");
Console.WriteLine($"{e2.Title} - {e2.Dept.Name}");
```

Answers:

Output is:

Dev - Testing

QA - Testing

→ Why: MemberwiseClone() creates shallow copy. Title (string) is independent, but Dept reference points to same Department object. Changing e2.Dept.Name also changes e1.Dept.Name.